

Instead, you can write your own version of `__str__()` and have it produce whatever string you want to help debug the code of your class. The general approach is to build a string that contains the values of any instance variables that you want to see and return that string to be printed. For example, we can add the following method to the `Vector` class from [Listing 9-8](#) to get information about any `Vector` object:

```
class Vector():
    --- snipped all previous methods ---
    def __str__(self):
        return 'This vector has the value (' + str(self.x) + ', ' + str(self.y) +
        ')
```

If you instantiate a `Vector`, you can then call the `print()` function and pass in a `Vector` object:

```
oVector = Vector(10, 7)
print(oVector)
```

Rather than just printing the memory address of the `Vector` object, you will get a nicely formatted report of the values of the two instance variables contained in the object:

```
This vector has the value (10, 7)
```

The main code in [Listing 9-9](#) creates a few `Vector` objects, does some vector math, and prints out the results of some `Vector` calculations.

File: Vectors/Main_Vectors.py

```
# Vector test code

from Vector import *

v1 = Vector(3, 4)
v2 = Vector(2, 2)
v3 = Vector(3, 4)

# These lines print Boolean or numeric values
print(v1 == v2)
```